Lesson 11 Notes: Tridiagonal and Square Band Matrices

1. Main idea of Lesson 11

Lesson 11 teaches a new way to store special matrices.

In earlier lessons, we stored useful values by **regions**:

Lessons	Matrix type	Main idea
Lessons 1–4	Diagonal matrix	Store only values where `i == j`.
Lessons 5–8	Lower triangular matrix	Store values where `i >= j`.
Lesson 9	Upper triangular matrix	Store values where `i <= j`.
Lesson 10	Symmetric matrix	Store one half and use the mirror rule `M[i][j] == M[j][i]`.

Lesson 11 changes the thinking model.

Instead of storing a triangle or half of a matrix, we store values by **diagonal lines**.

The main matrix type in this lesson is the **tridiagonal matrix**.

A tridiagonal matrix stores only three diagonals:

1. The main diagonal
2. The diagonal just below the main diagonal
3. The diagonal just above the main diagonal

Everything else is treated as zero.

---

2. What you should already know before Lesson 11

Before this lesson, you should already understand these ideas:

Concept	Meaning
Special matrix	A matrix with a pattern that helps save memory.
Structural zero	A zero that exists because of the matrix rule, not because it was stored.
1D array storage	Store only useful values in one array instead of storing the full matrix.
Matrix position	A value written as `M[i][j]`, where `i` is row and `j` is column.
1-based matrix indexing	Matrix rows and columns usually start from `1` in the formulas.
0-based array indexing	C and C++ arrays start from index `0`.
Direct formula	A formula that converts `(i, j)` into a 1D array index.

The important pattern is still the same:

1. Check whether `(i, j)` is a useful position.
2. If it is useful, calculate its 1D array index.
3. If it is not useful, treat it as zero.
4. Store only useful values.
5. Display the full matrix by printing stored values and fake zeros.

---

3. What is a tridiagonal matrix?

A **tridiagonal matrix** is a square matrix where useful values exist only on three diagonals.

Example:

```
1  2  0  0  0
3  4  5  0  0
0  6  7  8  0
0  0  9 10 11
0  0  0 12 13
```

This is a `5 x 5` tridiagonal matrix.

The useful values are:

```
1, 2,
3, 4, 5,
6, 7, 8,
9, 10, 11,
12, 13
```

The zeros are not stored.

They are structural zeros.

---

4. The three diagonals in a tridiagonal matrix

A tridiagonal matrix has three useful diagonal lines.

4.1 Main diagonal

The **main diagonal** goes from the top-left corner to the bottom-right corner.

Example positions:

```
M[1][1]
M[2][2]
M[3][3]
M[4][4]
M[5][5]
```

For every main diagonal position:

```
i == j
```

Another way to write this is:

```
i - j == 0
```

Example:

```
M[3][3]
i - j = 3 - 3 = 0
```

So `M[3][3]` is on the main diagonal.

4.2 Lower diagonal

The **lower diagonal** is the diagonal just below the main diagonal.

Example positions:

```
M[2][1]
M[3][2]
M[4][3]
M[5][4]
```

For every lower diagonal position:

```
i - j == 1
```

Example:

```
M[4][3]
i - j = 4 - 3 = 1
```

So `M[4][3]` is on the lower diagonal.

4.3 Upper diagonal

The **upper diagonal** is the diagonal just above the main diagonal.

Example positions:

```
M[1][2]
M[2][3]
M[3][4]
M[4][5]
```

For every upper diagonal position:

```
i - j == -1
```

Example:

```
M[2][3]
i - j = 2 - 3 = -1
```

So `M[2][3]` is on the upper diagonal.

5. The key test: `i - j`

The most important test in Lesson 11 is:

```
i - j
```

This tells us which diagonal a position belongs to.

Value of `i - j`	Meaning	Stored?
`1`	Lower diagonal	Yes
`0`	Main diagonal	Yes
`-1`	Upper diagonal	Yes
Anything else	Outside the tridiagonal band	No, return `0`

So the full rule is:

```
If i - j == 1, store in the lower diagonal area.
If i - j == 0, store in the main diagonal area.
If i - j == -1, store in the upper diagonal area.
Otherwise, the value is a structural zero.
```

Another compact way to write the rule is:

```
|i - j| <= 1
```

This means:

```
The row and column numbers can differ by at most 1.
```

If the difference is greater than `1`, the position is too far away from the main diagonal.

Then the value is zero.

---

6. Structural zeros in a tridiagonal matrix

A **structural zero** is a zero that exists because of the matrix rule.

In a tridiagonal matrix, any value outside the three diagonals is a structural zero.

Example:

```
M[4][1]
```

Check:

```
i - j = 4 - 1 = 3
```

The result is `3`.

But a tridiagonal matrix only stores positions where:

```
i - j == 1
or
i - j == 0
or
i - j == -1
```

Since `3` is none of these, `M[4][1]` is not stored.

So:

```
M[4][1] = 0
```

The program returns `0` without checking the array.

---

7. How many values does a tridiagonal matrix store?

For an `N x N` tridiagonal matrix:

```
Lower diagonal has N - 1 values.
Main diagonal has N values.
Upper diagonal has N - 1 values.
```

So the total number of stored values is:

```
(N - 1) + N + (N - 1)
```

Simplify:

```
N - 1 + N + N - 1 = 3N - 2
```

So:

```
Stored values = 3N - 2
```

---

8. Storage count example for `N = 5`

For a `5 x 5` tridiagonal matrix:

```
Lower diagonal = N - 1 = 5 - 1 = 4 values
Main diagonal  = N     = 5 values
Upper diagonal = N - 1 = 5 - 1 = 4 values
```

Total:

```
4 + 5 + 4 = 13
```

Using the formula:

```
3N - 2 = 3(5) - 2 = 15 - 2 = 13
```

A normal `5 x 5` matrix stores:

```
5 x 5 = 25 values
```

A tridiagonal `5 x 5` matrix stores only:

```
13 values
```

So it saves memory by avoiding the zeros outside the three diagonals.

---

9. Storage order used in this lesson

In this lesson, the tridiagonal matrix is stored in this order:

```
1. Lower diagonal first
2. Main diagonal second
3. Upper diagonal last
```

This is important because the index formulas depend on this order.

---

10. Example matrix and compact array

Consider this matrix:

```
1  2  0  0  0
3  4  5  0  0
0  6  7  8  0
0  0  9 10 11
0  0  0 12 13
```

10.1 Lower diagonal

The lower diagonal is:

```
3, 6, 9, 12
```

These are positions:

```
M[2][1]
M[3][2]
M[4][3]
M[5][4]
```

10.2 Main diagonal

The main diagonal is:

```
1, 4, 7, 10, 13
```

These are positions:

```
M[1][1]
M[2][2]
M[3][3]
M[4][4]
M[5][5]
```

10.3 Upper diagonal

The upper diagonal is:

```
2, 5, 8, 11
```

These are positions:

```
M[1][2]
M[2][3]
M[3][4]
M[4][5]
```

10.4 Final compact array

Since the storage order is lower, main, upper, the array becomes:

```
A = [3, 6, 9, 12, 1, 4, 7, 10, 13, 2, 5, 8, 11]
```

It is easier to see like this:

```
A = [ lower diagonal | main diagonal      | upper diagonal ]
A = [ 3 6 9 12       | 1 4 7 10 13       | 2 5 8 11       ]
```

---

11. Mapping table for the example

For the matrix:

```
1  2  0  0  0
3  4  5  0  0
```

```
0  6  7  8  0
0  0  9 10 11
0  0  0 12 13
```

The compact array is:

```
Index:  0  1  2  3  4  5  6  7  8  9 10 11 12
Value:  3  6  9 12  1  4  7 10 13  2  5  8 11
```

Mapping:

Matrix position	Value	Array index
`M[2][1]`	3	`A[0]`
`M[3][2]`	6	`A[1]`
`M[4][3]`	9	`A[2]`
`M[5][4]`	12	`A[3]`
`M[1][1]`	1	`A[4]`
`M[2][2]`	4	`A[5]`
`M[3][3]`	7	`A[6]`
`M[4][4]`	10	`A[7]`
`M[5][5]`	13	`A[8]`
`M[1][2]`	2	`A[9]`
`M[2][3]`	5	`A[10]`
`M[3][4]`	8	`A[11]`
`M[4][5]`	11	`A[12]`

---

12. Index formulas

Because the array has three sections, we need three formulas.

Target diagonal	Condition	Index formula
Lower diagonal	`i - j == 1`	`i - 2`
Main diagonal	`i - j == 0`	`(n - 1) + (i - 1)`

Target diagonal	Condition	Index formula
Upper diagonal	`i - j == -1`	`(2 * n - 1) + (i - 1)`

Use these formulas only after checking which diagonal the position belongs to.

---

13. Why the lower diagonal formula is `i - 2`

Lower diagonal positions are:

```
M[2][1]
M[3][2]
M[4][3]
M[5][4]
```

They are stored at the beginning of the array:

```
M[2][1] → A[0]
M[3][2] → A[1]
M[4][3] → A[2]
M[5][4] → A[3]
```

Formula:

```
index = i - 2
```

Check each position:

```
M[2][1]: index = 2 - 2 = 0
M[3][2]: index = 3 - 2 = 1
M[4][3]: index = 4 - 2 = 2
M[5][4]: index = 5 - 2 = 3
```

So the formula works.

---

14. Why the main diagonal formula is `(n - 1) + (i - 1)`

The main diagonal comes after the lower diagonal.

The lower diagonal has:

```
n - 1 values
```

So before the main diagonal starts, we must skip:

```
n - 1 array positions
```

Inside the main diagonal, the movement is:

```
i - 1
```

So the formula is:

```
index = (n - 1) + (i - 1)
```

For `n = 5`, the lower diagonal has:

```
n - 1 = 5 - 1 = 4 values
```

So the main diagonal starts at `A[4]`.

Mapping:

```
M[1][1] → A[4]
M[2][2] → A[5]
M[3][3] → A[6]
M[4][4] → A[7]
M[5][5] → A[8]
```

Example:

```
M[3][3]
index = (n - 1) + (i - 1)
index = (5 - 1) + (3 - 1)
index = 4 + 2
index = 6
```

So:

```
M[3][3] → A[6]
```

---

15. Why the upper diagonal formula is `(2 * n - 1) + (i - 1)`

The upper diagonal comes after both:

1. the lower diagonal
2. the main diagonal

Before the upper diagonal starts, we have already stored:

```
Lower diagonal = n - 1 values
Main diagonal  = n values
```

Total values before the upper diagonal:

```
(n - 1) + n = 2n - 1
```

So the upper diagonal starts at:

```
A[2n - 1]
```

Inside the upper diagonal, movement is:

```
i - 1
```

So the formula is:

```
index = (2 * n - 1) + (i - 1)
```

For `n = 5`:

```
2n - 1 = 2(5) - 1 = 10 - 1 = 9
```

So the upper diagonal starts at `A[9]`.

Mapping:

```
M[1][2] → A[9]
M[2][3] → A[10]
M[3][4] → A[11]
M[4][5] → A[12]
```

Example:

```
M[2][3]
index = (2 * n - 1) + (i - 1)
index = (2 * 5 - 1) + (2 - 1)
index = 9 + 1
index = 10
```

So:

```
M[2][3] → A[10]
```

---

16. Helper idea: find the array index

A helper function can return the array index for a stored position.

If the position is not stored, it can return `-1`.

Why `-1`?

Because valid array indexes are:

```
0, 1, 2, 3, ...
```

So `-1` is a useful signal that means:

```
This position is not stored.
```

Example helper function:

```c
int getTridiagonalIndex(int n, int i, int j) {
    if (i - j == 1) {
        return i - 2;
```

```
    }
    else if (i - j == 0) {
        return (n - 1) + (i - 1);
    }
    else if (i - j == -1) {
        return (2 * n - 1) + (i - 1);
    }

    return -1;
}
```

Important difference:

```
Index helper returns -1 for a non-stored position.
Get function returns 0 for a non-stored position.
```

Why?

Because `-1` is not the matrix value.

It is only a signal that there is no array index.

The actual matrix value at a structural zero position is `0`.

---

17. `getTridiagonal()` function

The `get` function returns the value at matrix position `M[i][j]`.

```
int getTridiagonal(int A[], int n, int i, int j) {
    if (i - j == 1) {
        return A[i - 2];
    }
    else if (i - j == 0) {
        return A[(n - 1) + (i - 1)];
    }
    else if (i - j == -1) {
        return A[(2 * n - 1) + (i - 1)];
    }

    return 0;
}
```

Explanation

The function checks the value of:

```
i - j
```

If `i - j == 1`, the position is on the lower diagonal.

So it returns:

```
A[i - 2]
```

If `i - j == 0`, the position is on the main diagonal.

So it returns:

```
A[(n - 1) + (i - 1)]
```

If `i - j == -1`, the position is on the upper diagonal.

So it returns:

```
A[(2 * n - 1) + (i - 1)]
```

Otherwise, the position is a structural zero.

So it returns:

```
0
```

---

18. Dry run setup

Assume:

```
n = 5
```

And:

```
A = [3, 6, 9, 12, 1, 4, 7, 10, 13, 2, 5, 8, 11]
```

This represents:

```
1  2  0  0  0
3  4  5  0  0
0  6  7  8  0
0  0  9 10 11
0  0  0 12 13
```

---

19. Dry run 1: Get `M[3][2]`

Call:

```
getTridiagonal(A, 5, 3, 2);
```

Values:

```
i = 3
j = 2
```

Step 1: Check which diagonal.

```
i - j = 3 - 2 = 1
```

Since the result is `1`, this position is on the lower diagonal.

Step 2: Use the lower diagonal formula.

```
index = i - 2
index = 3 - 2
index = 1
```

Step 3: Return the array value.

```
A[1] = 6
```

Final answer:

```
M[3][2] = 6
```

---

20. Dry run 2: Get `M[3][3]`

Call:

```
getTridiagonal(A, 5, 3, 3);
```

Values:

```
i = 3
j = 3
```

Step 1: Check which diagonal.

```
i - j = 3 - 3 = 0
```

Since the result is `0`, this position is on the main diagonal.

Step 2: Use the main diagonal formula.

```
index = (n - 1) + (i - 1)
index = (5 - 1) + (3 - 1)
index = 4 + 2
index = 6
```

Step 3: Return the array value.

```
A[6] = 7
```

Final answer:

```
M[3][3] = 7
```

---

21. Dry run 3: Get `M[2][3]`

Call:

```
getTridiagonal(A, 5, 2, 3);
```

Values:

```
i = 2
j = 3
```

Step 1: Check which diagonal.

```
i - j = 2 - 3 = -1
```

Since the result is `-1`, this position is on the upper diagonal.

Step 2: Use the upper diagonal formula.

```
index = (2 * n - 1) + (i - 1)
index = (2 * 5 - 1) + (2 - 1)
index = 9 + 1
index = 10
```

Step 3: Return the array value.

```
A[10] = 5
```

Final answer:

```
M[2][3] = 5
```

---

22. Dry run 4: Get `M[4][1]`

Call:

```
getTridiagonal(A, 5, 4, 1);
```

Values:

```
i = 4
j = 1
```

Step 1: Check which diagonal.

```
i - j = 4 - 1 = 3
```

The result is `3`.

This is not:

```
1, 0, or -1
```

So this position is outside the tridiagonal band.

Step 2: Return zero.

```
return 0
```

Final answer:

```
M[4][1] = 0
```

The function does not read the array because the matrix rule already tells us this value must be zero.

---

23. `setTridiagonal()` function

The `set` function stores a value at matrix position `M[i][j]`.

It uses the same formulas as `get`.

```c
void setTridiagonal(int A[], int n, int i, int j, int x) {
    if (i - j == 1) {
```

```
        A[i - 2] = x;
    }
    else if (i - j == 0) {
        A[(n - 1) + (i - 1)] = x;
    }
    else if (i - j == -1) {
        A[(2 * n - 1) + (i - 1)] = x;
    }

    // Otherwise, do nothing.
}
```

Explanation

The function receives:

Parameter	Meaning
`A[]`	The compact 1D array.
`n`	The matrix dimension.
`i`	Row number.
`j`	Column number.
`x`	Value to store.

The function stores `x` only if `(i, j)` belongs to one of the three diagonals.

If the position is outside the tridiagonal band, the function does nothing.

Why?

Because outside positions must remain structural zeros.

---

24. Dry run of `setTridiagonal()` for a stored position

Call:

```
setTridiagonal(A, 5, 4, 3, 99);
```

Values:

```
n = 5
i = 4
j = 3
x = 99
```

Step 1: Check which diagonal.

```
i - j = 4 - 3 = 1
```

Since the result is `1`, the position is on the lower diagonal.

Step 2: Use the lower diagonal formula.

```
index = i - 2
index = 4 - 2
index = 2
```

Step 3: Store the value.

```
A[2] = 99;
```

So `99` is stored at `A[2]`.

---

25. Dry run of `setTridiagonal()` for a structural zero

Call:

```
setTridiagonal(A, 5, 5, 2, 99);
```

Values:

```
i = 5
j = 2
x = 99
```

Step 1: Check which diagonal.

```
i - j = 5 - 2 = 3
```

The result is `3`.

This is not `1`, `0`, or `-1`.

So the position is outside the tridiagonal band.

Step 2: Do nothing.

The value `99` is ignored.

Why?

Because `M[5][2]` must remain zero in a tridiagonal matrix.

---

26. Displaying a tridiagonal matrix

The compact array stores only `3N - 2` values.

But when we display the matrix, we need to print the full `N x N` matrix.

So display uses nested loops.

```
void displayTridiagonal(int A[], int n) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cout << getTridiagonal(A, n, i, j) << " ";
        }
        cout << endl;
    }
}
```

Why display uses `getTridiagonal()`

For each position `(i, j)`, `getTridiagonal()` already knows what to do:

```
If the position is stored, return the stored value.
If the position is not stored, return 0.
```

So display can simply call `getTridiagonal()` for every position.

27. Why display is still `O(N²)`

Storage is small, but display must print the full visible matrix.

For an `N x N` matrix, the number of visible positions is:

```
N²
```

So display must visit:

```
N² positions
```

That is why display takes:

```
O(N²)
```

Even though storage is only:

```
O(N)
```

---

28. Full C++ example program

```cpp
#include <iostream>
using namespace std;

int getTridiagonal(int A[], int n, int i, int j) {
    if (i - j == 1) {
        return A[i - 2];
    }
    else if (i - j == 0) {
        return A[(n - 1) + (i - 1)];
    }
    else if (i - j == -1) {
        return A[(2 * n - 1) + (i - 1)];
    }

    return 0;
}
```

```
void setTridiagonal(int A[], int n, int i, int j, int x) {
    if (i - j == 1) {
        A[i - 2] = x;
    }
    else if (i - j == 0) {
        A[(n - 1) + (i - 1)] = x;
    }
    else if (i - j == -1) {
        A[(2 * n - 1) + (i - 1)] = x;
    }
}

void displayTridiagonal(int A[], int n) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cout << getTridiagonal(A, n, i, j) << " ";
        }
        cout << endl;
    }
}

int main() {
    int n = 5;
    int size = 3 * n - 2;
    int A[13] = {0};

    setTridiagonal(A, n, 1, 1, 1);
    setTridiagonal(A, n, 1, 2, 2);

    setTridiagonal(A, n, 2, 1, 3);
    setTridiagonal(A, n, 2, 2, 4);
    setTridiagonal(A, n, 2, 3, 5);

    setTridiagonal(A, n, 3, 2, 6);
    setTridiagonal(A, n, 3, 3, 7);
    setTridiagonal(A, n, 3, 4, 8);

    setTridiagonal(A, n, 4, 3, 9);
    setTridiagonal(A, n, 4, 4, 10);
    setTridiagonal(A, n, 4, 5, 11);

    setTridiagonal(A, n, 5, 4, 12);
    setTridiagonal(A, n, 5, 5, 13);

    displayTridiagonal(A, n);
```

```
    return 0;
}
```

Expected output:

```
1 2 0 0 0
3 4 5 0 0
0 6 7 8 0
0 0 9 10 11
0 0 0 12 13
```

---

29. Important note about array size

For a tridiagonal matrix, the array size is:

```
3n - 2
```

So in C++ we can calculate:

```
int size = 3 * n - 2;
```

For `n = 5`:

```
size = 3 * 5 - 2 = 13
```

So the array needs `13` cells.

In a dynamic memory version, we could write:

```
int *A = new int[3 * n - 2]{};
```

The `{}` initializes all values to zero.

At the end, we must release the memory:

```
delete[] A;
```

---

30. Complexity of tridiagonal matrix operations

Operation	Time complexity	Space complexity	Reason
Storage	—	`O(N)`	Stores only `3N - 2` values.
`getTridiagonal()`	`O(1)`	—	Uses direct condition checks and formulas.
`setTridiagonal()`	`O(1)`	—	Uses direct condition checks and formulas.
`displayTridiagonal()`	`O(N²)`	—	Prints every visible matrix position.

Why storage is `O(N)`

The matrix stores:

```
3N - 2 values
```

When complexity ignores constants and smaller terms, this becomes:

```
O(N)
```

So a tridiagonal matrix is much more memory-efficient than a full matrix.

A full matrix stores:

```
N² values
```

A tridiagonal matrix stores:

```
3N - 2 values
```

So storage changes from:

```
O(N²) to O(N)
```

---

31. Comparison with earlier matrices

Matrix type	Stored condition or rule	Stored values
Diagonal	`i == j`	`N`
Lower triangular	`i >= j`	`N(N + 1) / 2`
Upper triangular	`i <= j`	`N(N + 1) / 2`
Symmetric	Store one triangular half and mirror	`N(N + 1) / 2`
Tridiagonal	`i - j == 1`, `0`, or `-1`	`3N - 2`

The tridiagonal matrix stores more than a diagonal matrix, but much less than a triangular matrix for large `N`.

---

32. Square band matrices

A **square band matrix** is a general version of a tridiagonal matrix.

A tridiagonal matrix stores:

```
1 lower diagonal
1 main diagonal
1 upper diagonal
```

A square band matrix can store:

```
k lower diagonals
1 main diagonal
k upper diagonals
```

So a tridiagonal matrix is a square band matrix where:

```
k = 1
```

---

33. Example of a wider square band matrix

Example pattern:

```
x x x 0 0 0
x x x x 0 0
x x x x x 0
0 x x x x x
0 0 x x x x
0 0 0 x x x
```

Here:

```
x = useful value
0 = structural zero
```

The useful values are still near the main diagonal.

But the band is wider than a tridiagonal matrix.

---

34. Square band matrix rule

For a tridiagonal matrix, the useful positions satisfy:

```
|i - j| <= 1
```

For a square band matrix with bandwidth `k`, the useful positions satisfy:

```
|i - j| <= k
```

Meaning:

```
The row and column numbers can differ by at most k.
```

If:

```
|i - j| > k
```

then the position is outside the band and is a structural zero.

---

35. Tridiagonal vs square band matrix

Feature	Tridiagonal matrix	Square band matrix
Number of lower diagonals	`1`	`k`
Number of main diagonals	`1`	`1`
Number of upper diagonals	`1`	`k`
Stored condition	`|i - j| <= 1`	`|i - j| <= k`
Shape	Thin band around the main diagonal	Wider band around the main diagonal
Structural zeros	Farther than one diagonal away	Farther than `k` diagonals away

---

36. Why square band matrices save memory

A full `N x N` matrix stores:

```
N² values
```

A square band matrix stores only values near the main diagonal.

If the matrix is large but the band is narrow, many values outside the band are zeros.

So storing the full matrix wastes memory.

Instead, we store only the band values.

This is the same idea as tridiagonal matrix storage, but with more diagonals.

---

37. Main lesson summary

Lesson 11 teaches that we can store special matrices by diagonals.

The tridiagonal matrix stores only:

```
Lower diagonal
Main diagonal
Upper diagonal
```

The key test is:

```
i - j
```

Use this table:

Test	Meaning	Action
`i - j == 1`	Lower diagonal	Use `A[i - 2]`
`i - j == 0`	Main diagonal	Use `A[(n - 1) + (i - 1)]`
`i - j == -1`	Upper diagonal	Use `A[(2 * n - 1) + (i - 1)]`
Anything else	Structural zero	Return `0` or do nothing

Storage size:

```
3N - 2
```

Storage complexity:

```
O(N)
```

Get and set complexity:

```
O(1)
```

Display complexity:

```
O(N²)
```

---

38. Practice checks

Practice 1

Position:

```
M[5][4]
```

Check:

```
i - j = 5 - 4 = 1
```

Result:

```
Lower diagonal
```

Practice 2

Position:

```
M[5][5]
```

Check:

```
i - j = 5 - 5 = 0
```

Result:

```
Main diagonal
```

Practice 3

Position:

```
M[4][5]
```

Check:

```
i - j = 4 - 5 = -1
```

Result:

```
Upper diagonal
```

Practice 4

Position:

```
M[5][2]
```

Check:

```
i - j = 5 - 2 = 3
```

Result:

```
Structural zero
```

So:

```
M[5][2] = 0
```

---

39. Exit criteria

After this lesson, you should be able to:

1. Explain what a tridiagonal matrix is.
2. Identify the lower, main, and upper diagonals.
3. Use `i - j` to classify a matrix position.
4. Explain why positions outside the three diagonals are structural zeros.
5. Calculate the storage size `3N - 2`.
6. Build the compact 1D array in lower-main-upper order.

7. Use the correct index formula for each diagonal.
8. Write or understand `getTridiagonal()`.
9. Write or understand `setTridiagonal()`.
10. Explain why display is still `O(N²)`.
11. Explain how a square band matrix generalizes a tridiagonal matrix.

---

40. Final memory rule

A normal matrix stores everything:

```
N² values
```

A tridiagonal matrix stores only three diagonals:

```
3N - 2 values
```

So when most useful values are near the main diagonal, tridiagonal storage saves memory while still allowing direct `O(1)` access to stored values.